

Cover Page

***Submit to:**

Conference title:

The 19th Workshop on Combinatorial Mathematics and Computation Theory
March 29-30, National Sun Yat-Sen University, Kao-Hsiung, Taiwan

Conference co-chair:

Prof. Chang-Biau Yang
Department of Computer Science and Engineering
cbyang@cse.nsysu.edu.tw

***Paper title:**

An Efficient Algorithm for Finding Maximal Mean Subsequence

***Author Info.:**

Dr. Chun-Chao Yeh (corresponding author)

Assistant Professor
Department of Computer Science, National Taiwan Ocean University,
No. 2, Pei-Ning Road, Keelung, Taiwan 202

TEL: +886-2-2462-2192
FAX: +886-2-2462-3249
Email: ccyeh@cs.ntou.edu.tw

And,

Dr. Chun-Hsin Wu

Postdoctoral Fellow
Institute of Information Science, Academia Sinica,
Nankang, 115 Taipei, Taiwan

Tel: 886-2-2788-3799 ext. 2410
Fax: 886-2-2782-4814
Email: wuch@iis.sinica.edu.tw

An Efficient Algorithm for Finding Maximal Mean Subsequence

Chun-Chao Yeh

Department of Computer Science,
National Taiwan Ocean University,
No. 2, Pei-Ning Road, Keelung, 202, Taiwan
Email: ccyeh@cs.ntou.edu.tw

Chun-Hsin Wu

Institute of Information Science,
Academia Sinica,
Nankang, 115 Taipei, Taiwan
Email: wuch@iis.sinica.edu.tw

Abstract: - In this paper, we present an efficient method to find maximal mean subsequence (MMS) for a given sequence of non-negative number. When the minimal subsequence length is one, the MMS problem reduced to find a maximal number in the sequence. It can be solved easily. However, when the minimal subsequence length is arbitrary, then the problem become non-trivial. A simple version of solution will result in complexity of $O(n^2)$, for a sequence consists of n elements. In this paper we propose a more sophisticate algorithm to solve the problem with complexity of $O(Ln)$, where L is a given constant number as minimal subsequence length for the problem. Furthermore, we employ an efficient filtering scheme, named forward/backward cancellation, to accelerate search among all possible candidate subsequence. Consequently, the proposed algorithm can find the solution much efficiently.

Keywords: -maximal mean subsequence, MMS, forward/backward cancellation

1 Introduction

For a given sequence $A=(a_1, a_2, \dots, a_n)$, and a minimal subsequence length L , the maximal mean subsequence (MMS) problem, denoted by $MMS(A, L)$, is to find a continuous subsequence of A , $A(u,v)=(a_u, a_{u+1}, \dots, a_{u+v-1})$, so that mean of $A(u,v)$, denoted as $\bar{s}(u,v)$ is maximal. That is $\bar{s}(u,v)=\max\{\bar{s}(i,j) \mid 1 \leq i \leq j \leq n, L-1 \leq j-i \leq (n-1)\}$. It is worthy to note that the subsequence here we discuss in this context is for the “continuous” subsequence case only, while there is more general definition of subsequence [1]. For the case without the continuity constraint, the problem becomes trivial; under this case, it can be showed that the maximal mean subsequence can be determined by taking those first L largest elements in the sequence. For the following context, we refer the term of subsequence to the case of continuous subsequence only, except those with explicit notations. The MMS problem is intrinsically to find out a dominating subsequence (with length no less than L) with most significance in average. For example, some researchers use a similar scheme to find conserved regions in sequence alignment in bio-informatics research [2]. A simple solution can be done by exhausting search as shown in **Algorithm 1**, and result in complexity of $O(n^2)$, for a sequence consists of n elements. To our best knowledge, the problem has not been studied further before. In this paper we develop a more sophisticate search scheme to find optimal solutions. Instead of search all the possible value, we filter out

those infeasible solutions first and search with a sophisticated algorithm for those possible candidates only. There could be more than one subsequence with same maximal mean value. One of these maximal mean subsequences can be found among the subsequences with length between L and $2L-1$. Consequently, we can reduce search space largely, especially when L is far less than sequence size n . Furthermore, we employ an efficient filtering scheme to accelerate search among all possible candidate subsequences with length between L and $2L-1$. Combining these two techniques, the proposed algorithm can find the solution much efficiently. From intensive experiments, we show our algorithm greatly reduce search efforts than the one conducted by exhausted search.

Algorithm 1. Simple_MMS_quadratic

```

Procedure simple_mms_quadratic( L: integer, A: integer array)
{
/*  $\bar{s}(i,j) := (a_i + a_{i+1} + \dots + a_j) / k$ ;  $k=j-i+1$  */
  max, i,j,k: integer;
  max_i, max_j: integer;

  max := 0; max_i:=0; max_j:=0;
  /* search all subsequences (with length k from L to n) */
  for(k:= L to n) {
    for (i:= 1 to n-k+1) {
      j:=i+k-1;
      if (max >  $\bar{s}(i,j)$ )
        {max :=  $\bar{s}(i,j)$ ; max_i:=i; max_j:=j;}
    }
  }
  /*max mean subsequence is ( $a_{\max\_i}, a_{\max\_i+1}, \dots, a_{\max\_j}$ ) */
}

```

In the following section, we present some important properties of MMS. And, basing on these properties, in Section 3 we develop an efficient algorithm to solve the MMS problem. Also, we evaluate performance of the proposed algorithm with intensive experiments. The results show that the proposed algorithm can solve the problem efficiently. Finally, a concluding summary is given in the last section.

2 Some important properties of the MMS problem

For the following discussion, we define a MMS problem as $\text{MMS}(A,L)$, where A is an n -tuple array $(a_1, a_2, \dots, a_n)$, representing the input sequence, and L is the minimal subsequence length to be considered. Subsequence of A is denoted as $A(i,j) = (a_i, a_{i+1}, \dots, a_{i+j-1})$, $1 \leq i \leq j \leq |A|=n$. Length of subsequence $A(i,j)$ is defined as number of elements in it, that is $j-i+1$. Also we define two functions, $s(i,j)$ as sum of the subsequence $A(i,j)$ and $\bar{s}(i,j)$ as mean value of this subsequence. That is $s(i,j) = a_i + a_{i+1} + \dots + a_j$, and $\bar{s}(i,j) = s(i,j)/(j-i+1)$. For a given MMS problem $\text{MMS}(A,L)$, we would like to find a subsequence $A(u,v)$ resulting maximal mean. That is

$\bar{s}(u,v) = \max\{ \bar{s}(i,j) \mid 1 \leq i \leq j \leq n, L-1 \leq j-i \leq (n-1) \}$. Also, we add two extra boundary elements a_0 and a_{n+1} in A , with $a_0 = a_{n+1} = 0$, if necessary.

For a given n -number sequence, all subsequences' mean value can be listed and constructed as a triangle as shown in Figure 1. A simple algorithm to find the maximal mean subsequence is to search all over these mean value once and find the maximal one. This results in complexity $O(n^2)$. Further improvement can be made basing on following lemmas.

Lemma 1. For a given MMS problem $MMS(A,L)$, if $L=1$, then $MMS(A,L) = \max\{ a_k \mid \text{for all } k \}$.

Proof:

Let $y = \max\{ a_k \mid \text{for all } k \}$. Consider a subsequence $A(i,j)$. Mean value of $A(i,j) = \bar{s}(i,j) = s(i,j)/(j-i+1) = (a_i + a_{i+1} + \dots + a_j)/(j-i+1) \leq (y + y + \dots + y)/(j-i+1) = y(j-i+1)/(j-i+1) = y$. That is, $y \geq \bar{s}(i,j)$, for all $1 \leq i \leq j \leq n$. Q.E.D

Lemma 2 (Backward cancellation rule) Given a sequence of number, $A = (a_1, a_2, \dots, a_n)$, if $y \geq \bar{s}(k-1, j-1)$ and $y \geq \bar{s}(j, m)$, then $y \geq \bar{s}(k-1, m)$, for all k, m , $2 \leq k \leq j \leq m \leq n$.

Proof:

$$\begin{aligned}
& \text{Let } \bar{s}(k-1, j-1) = y - d_1, \bar{s}(j, m) = y - d_2, d_1, d_2 \geq 0. \\
& \bar{s}(k-1, m) = s(k-1, m)/(m-k+2) \\
& = ((a_{k-1} + \dots + a_{j-1}) + (a_j + \dots + a_m))/(m-k+2) \\
& = (s(k-1, j-1) + s(j, m))/(m-k+2) \\
& = ((j-k+1)\bar{s}(k-1, j-1) + (m-j+1)\bar{s}(j, m))/(m-k+2) \\
& = ((j-k+1)(y-d_1) + (m-j+1)(y-d_2))/(m-k+2) \\
& = (y(m-k+2) - ((j-k+1)d_1 + (m-j+1)d_2))/(m-k+2) \\
& = y - ((j-k+1)d_1 + (m-j+1)d_2)/(m-k+2) \leq y
\end{aligned}$$
Q.E.D

Lemma 3. (Forward cancellation rule) Given a sequence of number, $A = (a_1, a_2, \dots, a_n)$, if $y \geq \bar{s}(j+1, m+1)$ and $y \geq \bar{s}(k, j)$, then $y \geq \bar{s}(k, m+1)$, for all k, m , $1 \leq k \leq j \leq m \leq n-1$.

Proof:

$$\begin{aligned}
& \text{Let } \bar{s}(j+1, m+1) = y - d_1, \bar{s}(k, j) = y - d_2, d_1, d_2 \geq 0. \\
& \bar{s}(k, m+1) = s(k, m+1)/(m-k+2) \\
& = ((a_k + \dots + a_j) + (a_{j+1} + \dots + a_{m+1}))/m-k+2 \\
& = (s(k, j) + s(j+1, m+1))/(m-k+2) \\
& = ((j-k+1)\bar{s}(k, j) + (m-j+1)\bar{s}(j+1, m+1))/(m-k+2) \\
& = ((j-k+1)(y-d_2) + (m-j+1)(y-d_1))/(m-k+2) \\
& = ((y(m-k+2) - ((j-k+1)d_2 + (m-j+1)d_1))/(m-k+2) \\
& = y - ((j-k+1)d_2 + (m-j+1)d_1)/(m-k+2) \leq y
\end{aligned}$$
Q.E.D

Lemma 1 is a special case for $L=1$, we show that under such a case, the MMS problem $MMS(A,1)$ is equivalent to find the maximal number in the sequence A , which can be solved easily with complexity $O(n)$.

Lemma 2 and **Lemma 3** provide rules to skip comparisons with some subsequences, using some prior comparison results. Figure 2 shows the cases of applying Lemma 2 and Lemma 3 to the MMS problem represented in Figure 1. The line path $a \not\leq b \not\leq c$ in Figure 2, means if $y \geq a$ and $y \geq b$ then $y \geq c$.

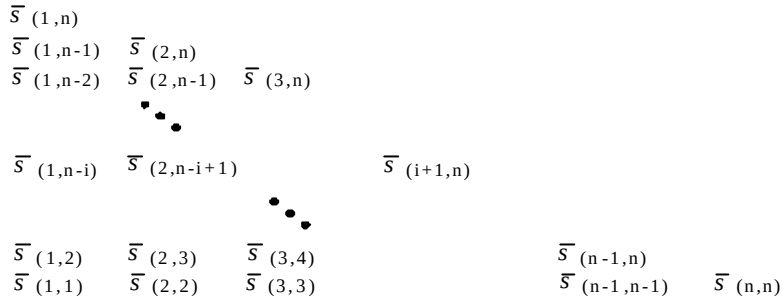
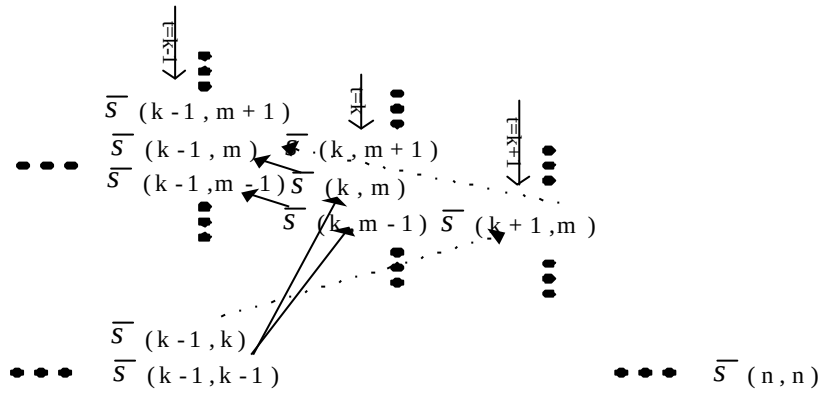
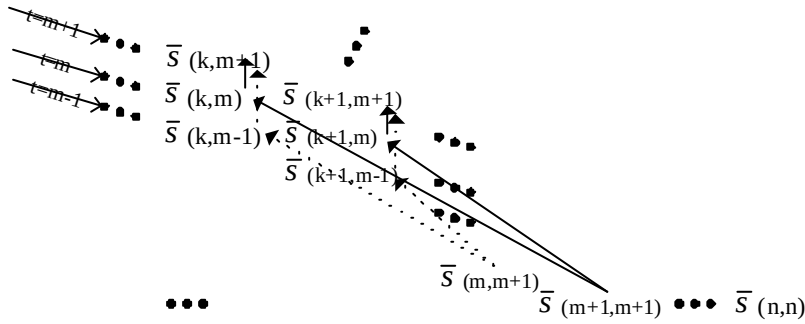


Figure 1. A triangle representation of all subsequence's mean value for a sequence consists of n elements



(a). Backward rule cancellation



(b) Forward rule cancellation

Figure 2. (a) Backward rule cancellation, and (b) Forward rule cancellation

3. Proposed algorithms to the MMS problem

3.1 Linear-time algorithm for the MMS problem

The MMS problem can be solved by a simple algorithm such as **Algorithm 1** mentioned in previous section.

In **Algorithm 1**, we search all possible value of $\bar{s}(i,j)$, for $1 \leq i \leq j \leq n$, $L-1 \leq j-i \leq (n-1)$, and results in $O(n^2)$ time complexity. Based on explored properties stated in Section 2, we show in this section that we can reduce the $O(n^2)$ search space to $O(L \times n)$. Consequently a linear-time algorithm is possible for the MMS problem.

Lemma 4. Given a sequence of number, $A = (a_1, a_2, \dots, a_n)$, (a) if $y \geq a_{k-1}$ and $y \geq \bar{s}(k,m)$, then $y \geq \bar{s}(k-1,m)$, for all k,m , $2 \leq k \leq m \leq n$. (b) if $y \geq a_{m+1}$ and $y \geq \bar{s}(k,m)$, then $y \geq \bar{s}(k,m+1)$, for all k,m , $1 \leq k \leq m \leq n-1$.

Proof:

(a) can be obtained from Lemma 2 by setting $j=k$. Similarly, (b) can be obtained from Lemma 3 by setting $j=m$. Q.E.D

Lemma 5. Given a sequence of number, $A = (a_1, a_2, \dots, a_n)$, and an integer L , if $y \geq \bar{s}(i,j)$, for all i,j , $1 \leq i < j \leq n$, $0 \leq L-1 \leq j-i < 2L-1$, then $y \geq \bar{s}(i,j)$, for all i,j , $L-1 \leq j-i < 3L-1$.

Proof:

Case 1. For all $L-1 \leq j-i < 2L-1$, we have $y \geq \bar{s}(j,i)$, by assumption.

Case 2. For all $2L-1 \leq j-i < 3L-1$, since $2L-1 \leq j-i < 3L-1$, we have $L-1 \leq (j-L)-i < 2L-1$ and $1 \leq i < j-L \leq n$. By assumption, we get $y \geq \bar{s}(i,j-L)$. Similarly, since $j-(j-L+1)=L-1$ and $1 \leq j-L+1 < j \leq n$, we get $y \geq \bar{s}(j-L+1,j)$ as well. From Lemma 3, we know if we have $y \geq \bar{s}(i,j-L)$ and $y \geq \bar{s}(j-L+1,j)$, then we have $y \geq \bar{s}(i,j)$. Q.E.D

Theorem 1 Given a sequence of number, $A = (a_1, a_2, \dots, a_n)$, and an integer L , if $y \geq \bar{s}(i,j)$, for all i,j , $1 \leq i < j \leq n$, $0 \leq L-1 \leq j-i < 2L-1$, then $y \geq \bar{s}(i,j)$, for all i,j , $1 \leq i < j \leq n$, $L-1 \leq j-i < n$.

Proof:

We prove this by induction. First, from Lemma 5 we have if $y \geq \bar{s}(i,j)$, for all i,j , $1 \leq i < j \leq n$, $0 \leq L-1 \leq j-i < 2L-1$, then $y \geq \bar{s}(i,j)$, for all i,j , $L-1 \leq j-i < 3L-1$. Then, Assume $y \geq \bar{s}(i,j)$, for all i,j , $1 \leq i < j \leq n$, $L-1 \leq j-i < kL-1$, $k \geq 3$. Consider for all $\bar{s}(i,j)$, $1 \leq i < j \leq n$, $kL-1 \leq L-1 \leq j-i < (k+1)L-1$. Since $(k-1)L-1 \leq (j-L)-i < kL-1$ and $1 \leq i < j-L \leq n$. By assumption, we get $y \geq \bar{s}(i,j-L)$. Similarly, since $j-(j-L+1)=L-1$ and $1 \leq j-L+1 < j \leq n$, we get $y \geq \bar{s}(j-L+1,j)$ as well. Consequently, we have both $y \geq \bar{s}(i,j-L)$ and $y \geq \bar{s}(j-L+1,j)$. Again, from Lemma 3, we have $y \geq \bar{s}(i,j)$, for $kL-1 \leq L-1 \leq j-i < (k+1)L-1$. Consequently, we have $y \geq \bar{s}(i,j)$, for $1 \leq i < j \leq n$, $L-1 \leq L-1 \leq j-i < pL-1$, $p \geq 2$. Thus we show $y \geq \bar{s}(i,j)$, for all $1 \leq i < j \leq n$, $L-1 \leq j-i < n$. Q.E.D

Theorem 1 shows that there exists one subsequence with maximal mean value among the subsequences with length less or equal to $2L-1$ (when n is greater than $2L-1$). As a result, we can modify **Algorithm 1** to **Algorithm 2** by changing only the searching subsequences from all to those subsequences with length no greater than $2L-1$ (The outer for loop.). Complexity of Algorithm 2 is $O(Ln)$.

Algorithm 2. Simple_MMS_linear

Procedure simple_mms_linear(L: integer, A: integer array)

```
{
    /*  $\bar{s}(i,j) := (a_i + a_{i+1} + \dots + a_j) / k$ ;  $k=j-i+1$  */
    max, i,j,k: integer;
    max_i, max_j: integer;

    max := 0; max_i := 0; max_j := 0;
```

```

/* search only those subsequences with length k from L to 2L-1 */
for(k:= L to min(2L-1,n)) {
    for (i:= 1 to n-k+1) {
        j:=i+k-1;
        if (max >  $\bar{s}(i,j)$ ) {max :=  $\bar{s}(i,j)$ ; max_i:=i; max_j:=j;}
    }
}
/* max mean subsequence is ( $a_{\max_i}, a_{\max_i+1}, \dots, a_{\max_j}$ ) */
}

```

3.2 Further improvement with forward and backward cancellation rules

Further improvement can be achieved by applying backward cancellation rule (Lemma 2) and forward cancellation rule (Lemma 3). First we consider all the possible subsequences (with length no less than L). Using the representation of Figure 1, all subsequence mean value can be partitioned into two subsets A and B as shown in Figure 3. The boarder line between A and B is $\bar{s}(i,v)$, $1 \leq i \leq u$, $v-u=L-1$, where $\bar{s}(u,v)$ is mean value of subsequence from u to v , which is assumed to be maximum among all subsequence with length equal to L . That is $A = \{ \bar{s}(i,j) \mid i \leq u, j \leq v \}$. B is complement set of A.

Apply a simple version of backward cancellation rule (as shown in Lemma 4 (a)) to elements in set A, line by line (vertical line shown in area A in Figure 3), for each line i containing elements $\bar{s}(i,j)$, $i+L-1 \leq j \leq v$, i from u back to 1. Assume a value y is no less than all elements in the vertical lines from u back to k , that is $y = \bar{s}(i,j)$, $i+L-1 \leq j \leq v$, $k \leq i \leq u$. If $y \geq a_{k-1}$, then we know, from Lemma 4, y is no less than all elements in the line $k-1$ except the element with length L starting from $k-1$ that is $\bar{s}(k-1, k+L-2)$. Since we backward search from line u which containing single element $\bar{s}(u,v)$, we know that $y = \bar{s}(u,v) \geq \bar{s}(k-1, k+L-2)$. Consequently if $y \geq a_{k-1}$, we can skip all the elements in line $k-1$ in the area A. Similarly, apply forward cancellation rule to elements in set B, line by line (diagonal line shown in area B in Figure 3), for each line j containing elements $\bar{s}(i,j)$, $1 \leq i \leq j-L+1$, j from $v+1$ to n . Again, if a value y is no less than all elements in the diagonal lines from $v+1$ to m , (that is $y = \bar{s}(i,j)$, $1 \leq i \leq j-L+1$, $v+1 \leq j \leq m$) and $y \geq a_{m+1}$, then y is no less than all elements in line $m+1$ except the element with length L ending at $m+1$ that is $\bar{s}(m-L+2, m+1)$. Since $y \geq \bar{s}(u,v)$, we have $y = \bar{s}(m-L+2, m+1)$ as well. As a result for all elements on the lines in area B, if $y \geq a_{m+1}$, we can skip all the elements in line $m+1$.

Consequently, for the case that only elements with subsequence length between L and $2L-1$ are concerned as in **Algorithm 2**, a more efficient algorithm can be modified from **Algorithm 2** by adding above forward/backward cancellation rule to filter out unlikely elements. **Algorithm 3** below describes details of the algorithm.

Algorithm 3. FB_cancellation_MMS_linear

Procedure FB_cancellation_mms_linear(L: integer, A: integer array)

```

{
  /*  $\bar{s}(i,j) := (a_i + a_{i+1} + \dots + a_j) / k; k=j-i+1$  */
  max, i,j,k: integer;
  max_i, max_j: integer;

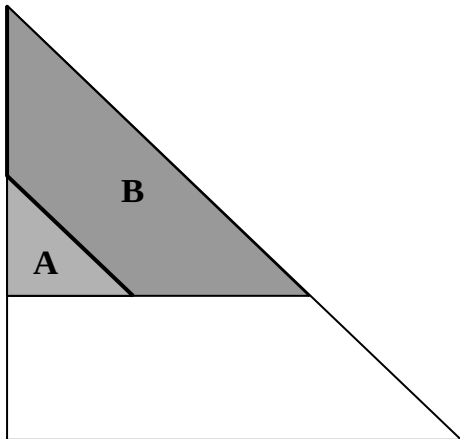
  find max mean subsequence A(u,v) with length equal to L (that is  $u-v+1=L$ ).
  max :=  $\bar{s}(u,v)$ ; max_i :=u; max_j :=v;

  /* search only those subsequences with length k from L to 2L-1 (the candidates) */
  /* 1. apply backward rule to search those candidates in area A */
  for (i:= u-1 down to 1) {
    if (max <  $a_i$ ) { /* compare vertical line i */
      for (j= i+L to min(i+ 2L-2, v) ) {
        if (max >  $\bar{s}(i,j)$ ) {max :=  $\bar{s}(i,j)$ ; max_i:=i; max_j:=j;}
      }
    }
  }

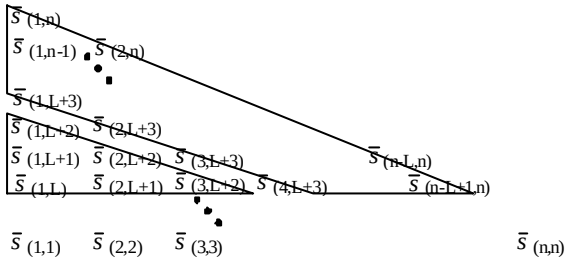
  /* 2. apply forward rule to search those candidates in area B */
  for (j:= v+1 to n) {
    if (max <  $a_j$ ) { /* compare diagonal line i */
      for (i= max(1,j-2L+2) to j-L ) {
        if (max >  $\bar{s}(i,j)$ ) {max :=  $\bar{s}(i,j)$ ; max_i:=i; max_j:=j;}
      }
    }
  }

  /* max mean subsequence is ( $a_{\max\_i}, a_{\max\_i+1}, \dots, a_{\max\_j}$ ) */
}

```



(a) Partition of all elements into two sets A, B, applying backward cancellation rule to elements in set A, and forward cancellation to elements in set B.



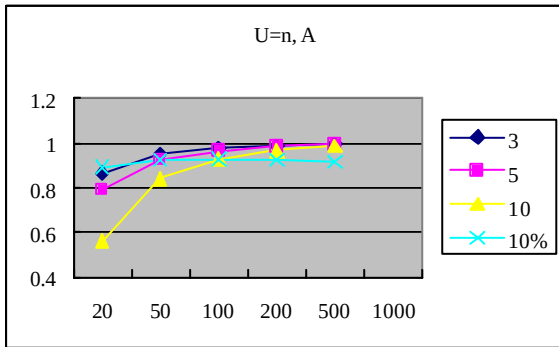
(b) Example of the partition, assume $\bar{s}(3,L+2)$ is max. of all subsequences with length L .

Figure 3. Partition of all elements into two sets A, B for forward/backward cancellation process

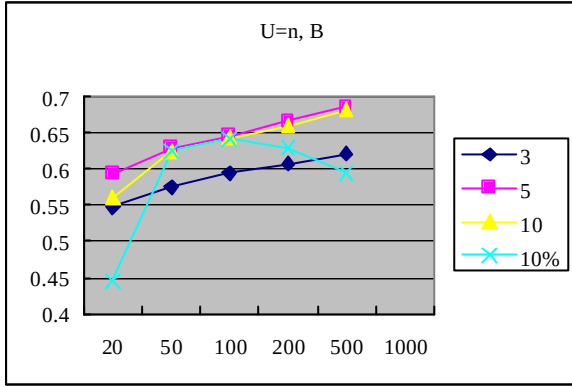
It is worthy to note that we apply a simple version of cancellation rule (Lemma 4) to the algorithm only. More elements might be able to be cancelled when a more complicate checking rule as Lemma 2 and Lemma 3 stated is applied. Nonetheless, this will generate more computing cost for the checking. From our experiments we found that in general, it works well when only a simple version of cancellation rule is applied.

3.3 Performance evaluation

To evaluate efficiency of the proposed algorithm, we make some experiments with computer simulations. We would like to know how much elements could be skipped (cancelled) during the maximum search. Three parameters are concerned, sequence length n ($n=20,50,100,200$, and 500), minimal subsequence length L ($L=3, 5, 10$, and $n/10$), and the value range of each element in the sequence $U(1-5, 1-10, 1-n/2, 1-n)$. Figure 4 and Figure 5 show parts of results.

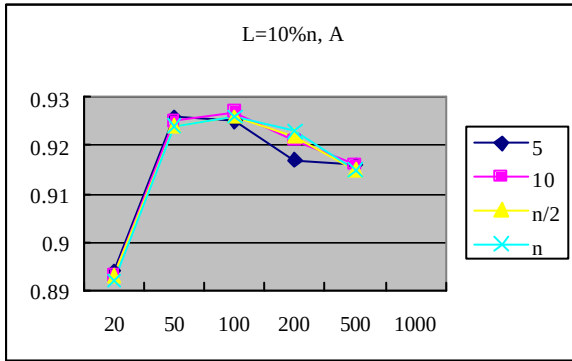


(a) Compound improvement comparing with exhausted search (Algorithm 1 vs. Algorithm 3)(percentage of skipped elements. Value rang: $1-n$; minimal subsequence length is 3, 5, 10, and $n/10$)

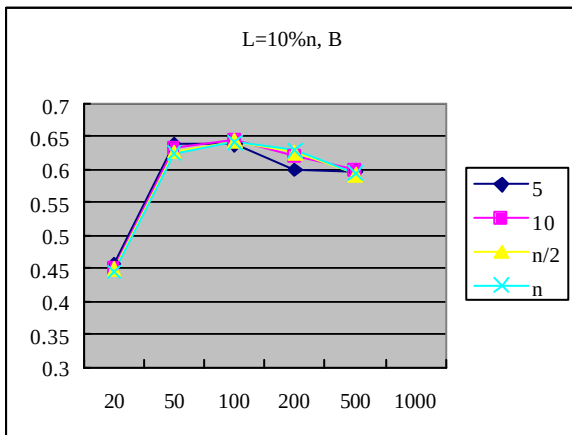


(b) Further improvement by forward /backward cancellation (Algorithm 2 vs. Algorithm 3) (percentage of skipped elements. Value rang: 1-n; minimal subsequence length is 3, 5, 10, and $n/10$)

Figure 4. Performance comparison (minimal subsequence length effects)



(a) Compound improvement comparing with exhausted search (Algorithm 1 vs. Algorithm 3)(percentage of skipped elements. Value rang: 5, 10, $n/2$, and n ; minimal subsequence length is $n/10$)



(b) Further improvement by forward /backward cancellation (Algorithm 2 vs. Algorithm 3) (percentage of skipped elements. Value rang: 5, 10, $n/2$, and n ; minimal subsequence length is $n/10$)

Figure 5. Performance comparison (Value range effects)

4. Concluding remarks

In this paper, we present an efficient method to find maximal mean subsequence (MMS) for a given sequence of non-negative number. A simple version of solution will result in complexity of $O(n^2)$ using exhausted search, for a sequence consists of n elements. We show that the problem can be solved efficiently by skipping those impossible candidates. First, we show only subsequences with length between L and $2L-1$ need to be checked. This important property reduce large of search space from $O(n^2)$ to $O(Ln)$. Then we further show that more elements can be skipped using the proposed forward /backward cancellation rules. The later can skip over fifty up to seventy percentage of elements (concerned in Algorithm 2). Also, from our simulation results, we found that compound performance improvement (algorithm 1 vs. algorithm 3) is quite large, especial when sequence length n is relatively much larger minimal subsequence length L . Under such a situation, over 96% of elements can be skipped.

Reference

- [1] Kenneth H. Rosen, "Discrete Mathematics and Its Applications," McGraw-Hill, 2000.
- [2] Kun-Mao Chao, "finding conserved regions in sequence alignment," lecture in Summer Institute on Bioinformatics, 2001, IIS/Sinica, Taipei, Taiwan,
"http://www.iis.sinica.edu.tw/~hil/summer/chao/chao.files/frame.htm"